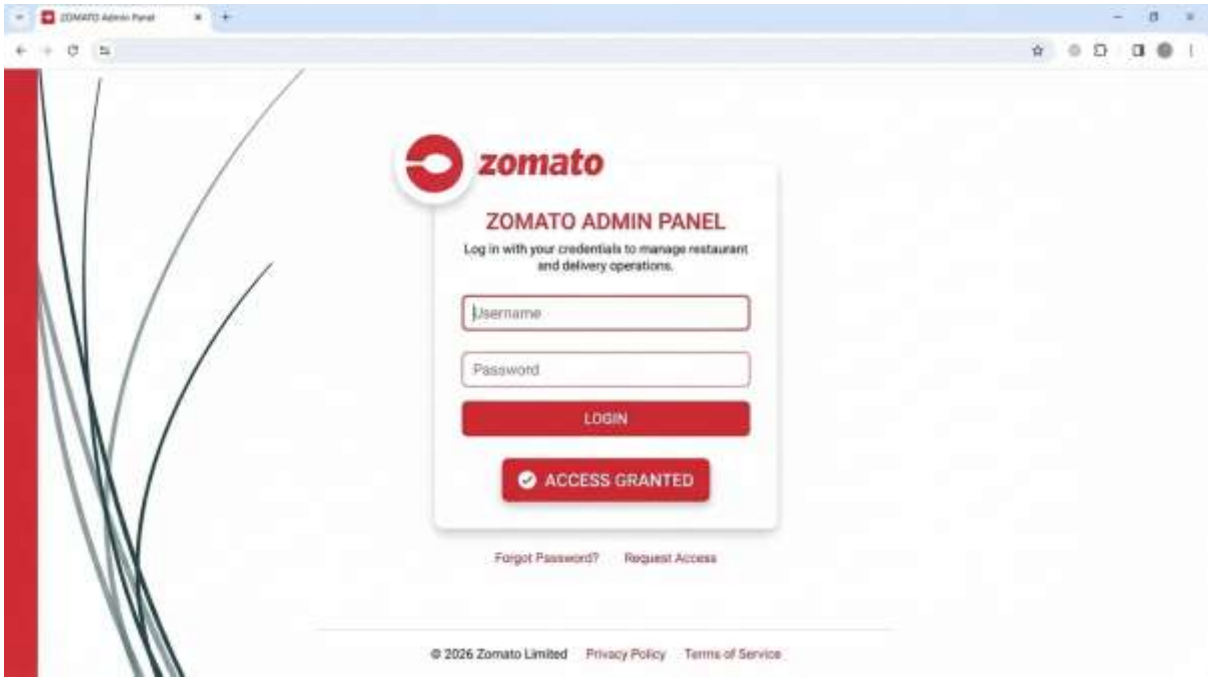
1.

Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

ANS:-



5.

Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

ANS:-

5 Common Access Control Mistakes in Real-World Apps

1. Insecure Direct Object References (IDOR)

- **The Mistake:** Allowing users to access or modify sensitive data (such as viewing another user's Flipkart order history or private Instagram

photos) simply by changing an ID or user number in the URL or API request.

- **Fix:** Implement robust server-side authorization checks to verify that the currently authenticated user owns or has explicit permission to access the requested resource.

2. Missing Function-Level Access Control

- **The Mistake:** Forgetting to restrict administrative or privileged API endpoints, allowing regular users or low-privileged accounts to execute sensitive actions (like deleting items or changing user roles) by guessing or directly calling the backend route.
- **Fix:** Enforce strict Role-Based Access Control (RBAC) middleware or permission checks on every single backend API endpoint and controller action.

3. Mass Assignment (Parameter Tampering)

- **The Mistake:** Automatically binding incoming user request data directly to database objects, allowing attackers to inject restricted fields (such as is_admin=true or changing product prices during checkout) into forms.
- **Fix:** Use explicit allowlists or Data Transfer Objects (DTOs) to define precisely which fields a user is allowed to modify.

4. Relying Solely on Client-Side Controls

- **The Mistake:** Hiding administrative buttons, edit features, or premium options in the frontend UI while leaving the underlying backend APIs completely unprotected against direct HTTP requests.
- **Fix:** Treat the client application as entirely untrusted and validate all access permissions and business logic on the server side.

5. Improper Session Invalidation on Logout or Privilege Escalation

- **The Mistake:** Failing to properly destroy session tokens or server-side session states when a user logs out or changes permissions, allowing old tokens to remain active.
- **Fix:** Invalidate session tokens on the server side and clear authentication cookies immediately upon logout or changes in user privileges.